“Smart Apartment Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Pamarthy Tejaswini **2520030011**

Sagi Sahasra Srihasa **2520030008**

Under the Esteemed Guidance of

Dr Spandana S
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Smart Apartment Management System "** is a record of Bonafide work of **Pamarthy Tejaswini - 2520030011, Sagi Sahasra Srihasa - 2520030008** submitted inpartial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Pamarthy Tejaswini – 2520030011

Sagi Sahasra Srihasa – 2520030008

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled**"Smart Apartment Management System"** is a bonafide work done and submitted by **Pamarthy Tejaswini - 2520030011, Sagi Sahasra Srihasa - 2520030008** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who have assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr Spandana S ,Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the ***Department for Computer Science Engineering/ Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Pamarthy Tejaswini – 2520030011

Sagi Sahasra Srihasa – 2520030008

Abstract

The Smart Apartment Management System is a web-based application designed to organize common activities within a residential society. It provides separate interfaces for residents and administrators, bringing features such as resident information, payment records, visitor management, complaints, amenities, notices, polls, and notifications into one system. Residents can use the portal to access society services and submit information, while administrators can manage related records through dedicated pages.

The application uses React for the front end, Node.js and Express.js for the back end, and MySQL for data storage. The front end communicates with the server through API requests. Authentication features support separate resident and administrator login flows, with passwords checked using bcrypt and JSON Web Tokens created after successful login.

The portal aims to make society-related information and tasks easier to access and manage through a single application. Its modular structure groups features into separate pages, APIs, routes, and controllers, making the system easier to understand and extend. The project also provides practical experience in connecting a user interface, server-side application logic, and a relational database. Future improvements may include stronger server-side authorization, production deployment, and additional integrations.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	9-14
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	15-17
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	18-22
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	23-26
5	Design	5.2 ER Diagram / Database schema	27
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	28-31
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	32-34
8	Testing	8.1 Functional Testing of the Front End and Back End 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	35-39
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	40-42
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	43-46
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	47-49
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	48-50
13	References	- Books, tutorials, APIs, documentation sites used	51

14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	52 -

1. Introduction

1.1 Background of the Project

Residential societies and apartment communities need to coordinate many activities to keep daily operations organized. These activities include maintaining resident information, recording payments, monitoring visitor entries, resolving complaints, managing shared facilities, and communicating updates. As the number of residents and services grows, keeping this information accurate and accessible can become increasingly difficult.

In some societies, these tasks are handled through a mixture of paper registers, spreadsheets, phone calls, and messaging groups. Although these methods can support basic communication, information is often stored in different places. Administrators may need to search several records to respond to a request, while residents may have difficulty finding current information or following up on a service issue. Repeated manual updates can also lead to incomplete or inconsistent records.

The Society Management Portal is a web application developed to bring several of these activities together. It provides residents and administrators with role-specific workflows through a shared system. Residents can use the portal to access services and information relevant to them, while administrators can manage records and society activities through an administrative interface.

The application uses React for the frontend, Node.js and Express for backend services, and MySQL for persistent data storage. The frontend communicates with backend REST APIs, which process requests and work with the database. This structure separates the user interface, application logic, and data storage into distinct parts of the system.

1.2 Problem Statement

When society services are managed through separate records and communication channels, it can be difficult to maintain a clear and consistent view of residents, payments, visitors, complaints, bookings, notices, and polls. Residents may need to contact administrators through different channels to obtain information or request assistance. Administrators, in turn, may need to manually locate records, update multiple documents, and communicate the same information to several people.

These processes can make routine tasks harder to track. For example, a complaint may be reported in one channel while its status is recorded elsewhere. A visitor entry or amenity booking may be recorded in a register that is not readily available to other authorized users. Notices and poll information can become difficult to find after they have been shared.

The project addresses this problem by developing a centralized web portal for common society management activities. It aims to organize key records and workflows in one application, with separate access for residents and administrators. The portal provides a consistent way to manage information and make relevant services accessible through a web interface.

1.3 Scope of the Project

The scope of the project is the development of a web-based portal for residents and society administrators. The application includes a React frontend, a Node.js and Express backend, REST API endpoints, and a MySQL database. Users interact with the frontend, which sends requests to the backend; the backend processes these requests and reads or updates the corresponding database records.

The main functional areas covered by the portal include:

- **User access:** Resident registration and login, administrator login, and role-based access to relevant workflows.
- **Resident management:** Viewing and managing resident information, including account details and flat-related information.
- **Payment records:** Recording and viewing payment-related information associated with residents.
- **Visitor management:** Recording visitor information and associating visitor entries with residents.
- **Complaint management:** Submitting complaints and allowing their status to be managed.
- **Amenity management:** Maintaining information about shared amenities and managing resident bookings.
- **Notice management:** Publishing society notices for users to view.
- **Poll management:** Creating polls, adding options, and recording resident votes.
- **Notifications:** Creating and managing notifications associated with users.

The project focuses on these core society workflows and the data required to support them. The current implementation does not include live payment gateway processing, external accounting-system integration, or a push notification service such as Firebase Cloud Messaging. These capabilities can be considered as future enhancements. The portal also does not replace every possible society operation; it provides a software foundation for the specific functions included in the project.

1.4 Objectives

The objectives of the Society Management Portal are to:

1. Develop a centralized web application for common residential society management activities.
2. Provide separate, role-appropriate workflows for residents and administrators.
3. Organize resident information and service records in a MySQL database.
4. Support payment record management, visitor tracking, complaint handling, amenity bookings, notices, polls, and notifications.
5. Connect the frontend and backend through REST APIs.
6. Apply role-based access so users interact with functions appropriate to their role.
7. Reduce reliance on scattered records by making supported information available through a shared portal.
8. Build the application with distinct frontend, backend, and database components to support maintenance and future development.

1.5 Importance of the Application

A centralized portal can make common society information easier to manage and access. Instead of relying exclusively on separate registers or communication channels, administrators can use a shared application to maintain supported records. Residents can access relevant functions through a consistent interface.

The portal is also important because it brings several types of society activity into the same system. Resident records can be connected with associated payment information, visitor entries, complaints, amenity bookings, and poll participation. Notices and notifications provide additional ways to share information with users. Keeping these functions together can make it easier to review related information when carrying out routine administrative tasks.

Role-specific workflows help distinguish resident-facing activities from administrator tasks. Residents can interact with services such as complaints, bookings, and polls, while administrators can manage records and oversee society functions. This separation helps organize the application around the needs of its primary user groups.

From a technical perspective, the project demonstrates how a web application can be organized across a user interface, backend API, and relational database. The React frontend provides the interactive interface, the Express backend handles application requests, and MySQL stores structured records. This arrangement provides a foundation on which additional capabilities may be developed, subject to future requirements and implementation.

1.6 Target Users/Audience

The primary target users of the application are residents and society administrators.

Residents are users who live in the society and need access to relevant services and information. Depending on the available portal functions, they can use the application to view payment-related information, record or review visitor details, submit complaints, book shared amenities, participate in polls, and view notices or notifications. A single portal gives residents a consistent place to interact with these services.

Society administrators or management committee members are responsible for coordinating society operations. They can use the administrative workflows to manage resident information, payment records, visitor entries, complaints, amenities and bookings, notices, polls, and notifications. The portal is intended to support these tasks by organizing the associated records and actions in one application.

The project is also relevant to students, developers, and academic evaluators interested in web application engineering. It demonstrates the use of a frontend framework, backend REST APIs, role-based workflows, and relational data storage in a practical management application.

2. System Requirements

This chapter describes the hardware and software environment needed to develop, run, and demonstrate the Society Management Portal. The application has a React frontend, a Node.js and Express backend, and a MySQL database. During local development, these components can run on one computer. In a deployed environment, they may run on separate systems depending on hosting and usage requirements.

2.1 Hardware Requirements

The following configurations are suggested for development and academic demonstration. The actual requirements may vary depending on the operating system, the number of applications running at the same time, and the size of the database.

Component	Minimum suggested configuration	Recommended configuration
Processor	Dual-core processor	Quad-core processor or better
Memory	8 GB RAM	16 GB RAM or more
Available storage	10 GB free space for the application, dependencies, and database	20 GB or more
Display	1366 × 768 resolution	1920 × 1080 resolution
Input devices	Keyboard and pointing device	Keyboard and mouse or touchpad
Network	Internet connection for obtaining dependencies and accessing GitHub	Stable broadband connection

A computer used for development should be able to run a code editor, a browser, the frontend development server, the backend server, and MySQL. Eight gigabytes of memory is a practical baseline for running these tools together; additional memory can improve responsiveness when several applications are open.

A resident or administrator accessing the portal needs a device capable of running a modern web browser. A laptop or desktop computer is suitable for development and administrative tasks. The portal’s performance on mobile devices depends on the layout and browser support of the deployed frontend.

These specifications describe a local development and demonstration environment. They are not production hosting requirements. The project has not established production capacity through load testing, so server sizing for a live deployment would need to be determined from the expected number of users, traffic, storage needs, and availability requirements.

2.2 Software Requirements

The portal requires software for the frontend, backend, database, and development environment.

Software	Purpose
Windows, macOS, or Linux	Operating system for development or deployment
Modern web browser	Displays the resident and administrator interfaces
Node.js runtime	Runs the backend application and frontend development tools
npm	Installs and manages JavaScript dependencies
React	Builds the interactive frontend
Express.js	Provides backend routing and REST API functionality
MySQL Server	Stores user, resident, payment, visitor, complaint, amenity, booking, notice, poll, and notification data
`mysql2` package	Connects the Node.js backend to MySQL
Git	Tracks source-code changes
GitHub	Hosts and shares the project repository
Visual Studio Code	Provides an editor for project files

A browser is required to access the portal. The frontend and backend must be running and configured to communicate with one another. The backend must also be able to connect to a MySQL database containing the required application tables.

The software versions used for installation should be compatible with the project's dependency files and runtime configuration. The project's package manifests should be treated as the reference for installing JavaScript packages. Database connection settings must also match the MySQL environment used for the demonstration or deployment.

2.3 Development Tools and Frameworks

React

React is used to build the portal's user interface. It organizes the interface into reusable components, such as forms, tables, dashboards, and navigation elements. This component-based approach supports separate resident and administrator views while allowing common interface elements to be reused.

Node.js and Express.js

Node.js provides the runtime environment for the backend JavaScript application. Express.js is used to define routes and handle requests from the frontend. The backend exposes REST API endpoints for operations such as retrieving records, creating entries, and updating statuses.

MySQL and `mysql2`

MySQL stores the portal's structured information in related tables. These records include users and roles, residents, payments, visitors, complaints, amenities, bookings, notices, notifications, polls, poll options, and votes. The backend uses the `mysql2` package to communicate with the database.

Visual Studio Code

Visual Studio Code is used to view and edit the frontend and backend source files. Its integrated terminal can be used to install dependencies and run the project's development commands.

Git and GitHub

Git provides version control for tracking source-code changes. GitHub hosts the project repository and supports collaboration and access to the project files.

Browser and local development environment

A browser is used to access the running frontend and interact with the portal. During local development, the frontend, backend, and MySQL database may run on the same computer. The frontend sends requests to the backend API, and the backend reads or updates data in MySQL.

Together, these tools provide the environment needed to implement and demonstrate the portal's core workflows. They support interface development, API handling, database operations, source-code management, and manual inspection of backend requests.

3. Technology Stack

The Society Management Portal uses a web-based technology stack consisting of a frontend, backend, database, and version-control tools. The frontend provides the user interface, the backend processes application requests, and the database stores the information used by the portal.

3.1 Front-end

The frontend is developed using React, JavaScript, HTML, and CSS. React is used to build the portal's interactive interface from reusable components. These components support features such as forms, navigation, dashboards, and record displays.

HTML provides the structure of the pages, while CSS controls their layout and visual presentation. JavaScript handles user interactions and the exchange of information with the backend. React Router is used to navigate between the different views of the application.

The portal provides separate interfaces for residents and administrators. Residents can access functions such as viewing payment information, recording visitors, submitting complaints, booking amenities, participating in polls, and viewing notices. Administrators can manage the records and services available in the system.

3.2 Back-end

The backend is developed using Node.js with the Express.js framework. Node.js runs the server-side JavaScript code, while Express.js is used to define routes and handle requests from the frontend.

The backend provides REST API endpoints for the portal's main features, including authentication, residents, payments, visitors, complaints, amenities, bookings, notices, notifications, and polls. When a request is received, the backend processes it, communicates with the database when needed, and returns a response to the frontend.

The application has separate login workflows for residents and administrators. It uses role information to support access to the functions associated with each type of user. JSON Web Tokens (JWTs) are used in the authentication flow.

3.3 Database

MySQL is used as the database management system for the project. It stores the structured information needed by the portal, including user accounts, roles, resident details, payment records, visitor entries, complaints, amenities, bookings, notices, notifications, polls, poll options, and votes.

The database uses identifiers to associate related records. For example, payment, visitor, and complaint records are associated with residents; amenity bookings are associated with amenities and residents; and poll options and votes are associated with their respective polls.

The Node.js backend connects to MySQL using the `mysql2` package. Database operations are handled by the backend in response to API requests, so the frontend does not connect directly to the database.

3.4 Version Control

Git is used to track changes to the project's source code. Version control helps maintain a history of updates and makes it easier to review or restore earlier changes.

GitHub hosts the project repository and provides a shared location for the source code. It allows team members to access and collaborate on the project files.

3.5 APIs or External Services

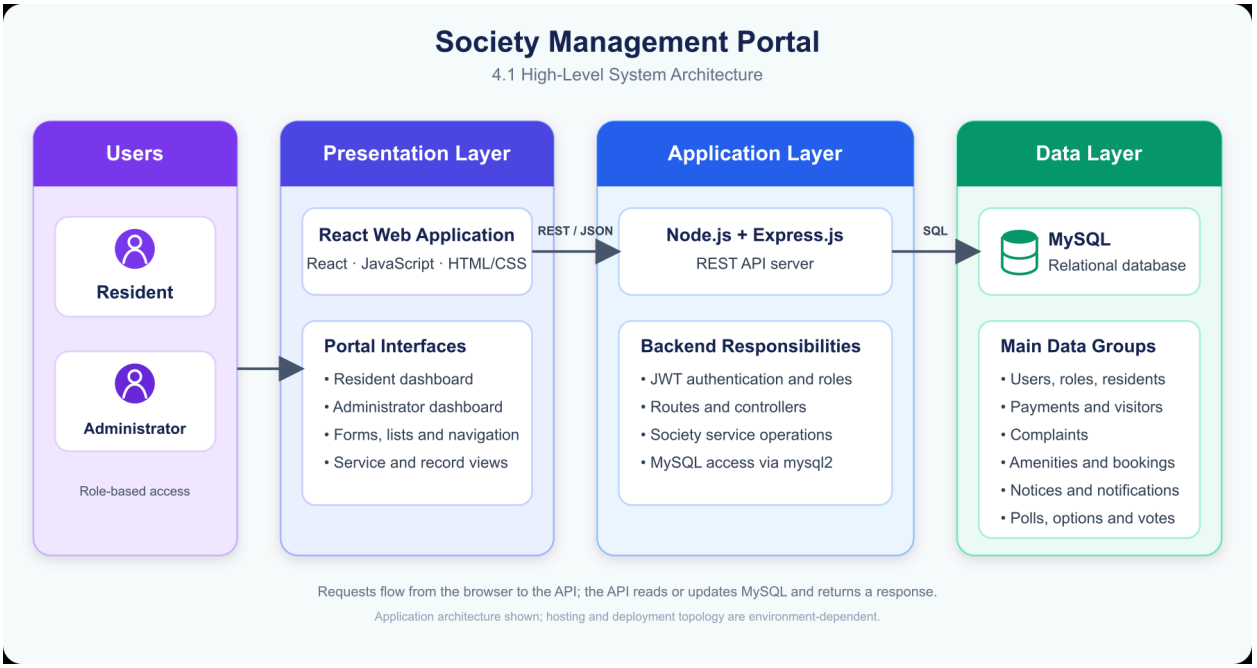
The project uses REST APIs developed with Express.js to connect the React frontend to the Node.js backend. These APIs support the portal's internal features, such as login, resident management, payment records, visitor records, complaint handling, amenity bookings, notices, notifications, and polls.

The frontend sends requests to the backend, and the backend returns responses, generally in JSON format. When data needs to be retrieved or updated, the backend communicates with the MySQL database using `mysql2`.

The project does not currently integrate third-party services such as Firebase Cloud Messaging or an external payment gateway. Payment functionality in the portal is for managing payment records; processing payments through an online gateway could be considered as a future enhancement.

4. System Architecture

4.1 high-level architecture diagram.



4.2 Description of Each Layer

The Society Management Portal is organized into three main application layers: the frontend, backend, and database. Each layer has a distinct responsibility. The frontend presents the portal to users, the backend processes application requests, and the database stores the information used by the system.

4.2.1 Front-end Layer

The frontend layer is the part of the application that residents and administrators interact with through a web browser. It is developed using React, JavaScript, HTML, and CSS. React components provide the interface elements, while HTML and CSS structure and style the pages. React Router supports navigation between the different views.

The portal provides role-specific interfaces. Residents can access functions such as payment information, visitor records, complaint submission, amenity bookings, polls, notices, and notifications. Administrators can use the relevant management views to oversee resident and society records.

When a user performs an action, such as opening a record or submitting a form, the frontend sends a request to the backend API. It then displays the response returned by the backend. The frontend does not connect directly to MySQL.

4.2.2 Back-end Layer

The backend layer is implemented using Node.js and Express.js. Node.js runs the server-side application, while Express.js handles routes and requests from the frontend. The backend exposes REST API endpoints for functions such as authentication, resident management, payments, visitors, complaints, amenities, bookings, notices, notifications, and polls.

Backend routes direct incoming requests to the appropriate controllers. The controllers handle application operations, such as retrieving records, adding information, or updating a status. They also communicate with the MySQL database when a request requires stored information.

The backend handles authentication and role information for resident and administrator workflows. JSON Web Tokens (JWTs) are used in the login flow. By processing requests on the server, the backend keeps database operations separate from the user interface.

4.2.3 Database Layer

The database layer uses MySQL to store the portal's structured data. It contains records for users, roles, residents, payments, visitors, complaints, amenities, amenity bookings, notices, notifications, polls, poll options, and poll votes.

Identifiers are used to connect related records. For example, payment, visitor, and complaint records are associated with residents. Amenity bookings are associated with both amenities and residents. Poll options and votes are associated with polls, and notifications are associated with users.

The backend accesses MySQL through the `mysql2` package. When the backend receives a request that needs data, it performs the appropriate database operation and returns the result to the frontend through the API. This keeps data access within the backend and allows the frontend

to work with API responses rather than direct database queries.

4.2.4 Interaction Between the Layers

The layers work together to complete each user action. The user interacts with the frontend in a browser. The frontend sends a request to an Express API endpoint. The backend processes the request and, when necessary, uses `mysql2` to read or update data in MySQL. The backend then returns a response, which the frontend displays to the user.

This separation makes each layer responsible for a clear part of the system: the frontend handles presentation and interaction, the backend handles request processing and application logic, and the database layer stores and retrieves application data.

4.3 Deployment Architecture

The project uses a layered application architecture consisting of a React frontend, a Node.js and Express backend, and a MySQL database. The supplied project materials identify the technologies and source repository, but do not document a cloud deployment or an automated CI/CD pipeline. Therefore, this section describes the local development and demonstration setup, followed by possible deployment options.

4.3.1 Local Development and Demonstration

For development or demonstration, the frontend, backend, and database can run in a local environment. The user opens the frontend in a web browser. The frontend sends requests to the Express API, and the backend connects to MySQL using the `mysql2` package.

The local environment requires Node.js and npm for the JavaScript applications, a MySQL server for data storage, and a browser for accessing the portal. The backend's database connection settings must match the MySQL instance used for the demonstration. This arrangement is suitable for development and academic presentation because the main application components can be run and examined without requiring a separate cloud environment.

4.3.2 Cloud Deployment

A cloud deployment is a possible future option; it is not identified as part of the current implementation. In such a deployment, the React frontend could be hosted as a web application, the Node.js and Express backend could run as a server-side service, and MySQL could be hosted separately as a database service.

The frontend would communicate with the backend through the REST API. The backend would connect to the database over a restricted network connection. A deployed version should use HTTPS and keep database credentials and other configuration values out of public source files. The database should only be accessible to the backend and authorized administrators.

The exact hosting provider and server capacity would depend on expected users, traffic, storage, availability needs, and budget. These requirements would need to be determined before a production deployment.

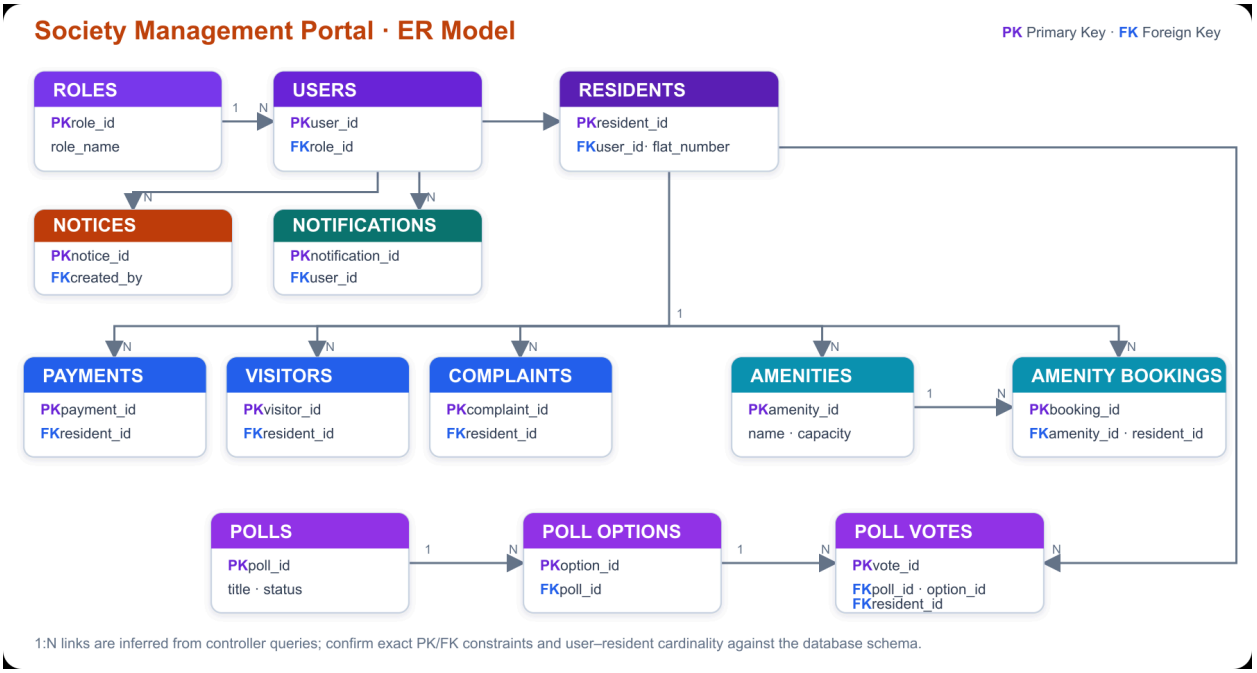
4.3.3 CI/CD Pipeline

Git and GitHub are used for version control and repository hosting. The supplied project materials do not identify a configured CI/CD pipeline. Consequently, automated build, test, and deployment steps should not be described as implemented features.

A CI/CD pipeline could be introduced as a future improvement. Such a pipeline could install project dependencies, build the frontend, run selected checks, and deploy approved changes to a hosting environment. Before adding automated deployment, the project would need a defined hosting target and a process for managing environment configuration and database updates.

5. Design

5.1 ER Diagram / Database schema



6. Implementation

The Society Management Portal was implemented as a web application with a React front end and a Node.js and Express back end connected to a MySQL database. The system separates resident and administrator workflows. Residents can access services such as payments, visitor management, complaints, amenities, notices, polls, and notifications. Administrators use a separate interface to manage these services and their associated records.

6.1 Module-wise Implementation

The application is divided into modules so that each area of society management has a focused set of screens and server-side operations.

Authentication module: This module provides separate login flows for residents and administrators. Resident registration collects user details and creates the corresponding resident record. Login credentials are checked by the back end before the user is directed to the appropriate area of the application.

Resident module: Residents can view their dashboard and profile, review payments, continue to payment checkout, register visitors, submit complaints, view amenity information, read notices, participate in polls, and check notifications. The front end provides pages for these activities, while the back end exposes matching API routes and controllers.

Administrator module: The administrator interface includes pages for the dashboard, residents, payments, visitors, complaints, amenities, notices, polls, and notifications. These pages support the society-management workflows available in the project.

Service modules: Amenities, complaints, notices, notifications, payments, polls, residents, and visitors are handled through separate back-end route and controller modules. This organization keeps the request-handling logic for each feature grouped by function and makes the application easier to maintain.

6.2 Front-end Logic (UI Rendering and State Management)

The front end is built with React. Individual pages and reusable interface components render the resident and administrator views. React Router is used to map application URLs to pages, including separate routes for login, dashboards, and feature screens.

The pages use React's `useState` and `useEffect` hooks to manage local interface state and load data. For example, state is used to hold form values, lists returned by the server, loading status, and error messages. Effects are used where a page needs to request data when it loads or when relevant values change.

Forms collect information such as login details, resident data, complaints, and visitor details. When a user submits a form, the page calls the relevant API function and updates its displayed state based on the response. This allows the interface to show current data and provide feedback when an operation succeeds or fails.

The application stores some login-related values in the browser's `localStorage`, including resident information and authentication tokens. The project uses page-level React state and browser storage; no Redux or other external global state-management library was identified in the supplied source.

6.3 API Integration

The front end communicates with the back end through HTTP API requests using JavaScript's native `fetch` function. The API functions are organized by feature in the front-end API directory. They send and receive JSON data and check the response status so that unsuccessful requests can be handled by the calling page.

During local development, the API base address is configured as `http://localhost:5000/api`. On the server, Express maps feature routes under this API path. Route handlers pass requests to controllers for operations involving residents, payments, visitors, complaints, amenities, notices, polls, and notifications. The back end uses MySQL for persistent data storage.

A typical data flow begins when a user opens a page or submits a form. The React page calls the relevant API function, which sends a request to the Express server. The server processes the request through the appropriate route and controller, interacts with the database where required, and returns a response. The front end then uses that response to update the page or display an error.

6.4 Authentication and Authorization

The project provides separate authentication flows for residents and administrators. During login, the server retrieves the relevant user information and compares the submitted password with the stored password hash using `bcrypt`. For resident login, the query also retrieves the resident record associated with the user. For administrator login, the system checks that the account has the administrator role.

After successful login, the server creates a JSON Web Token (JWT) that expires after one day. The front end stores login-related values in `localStorage` and presents separate resident and administrator pages. This supports identifying the type of user who has signed in and keeping the corresponding user information available to the interface.

In the supplied project source, a shared back-end middleware for verifying JWTs and enforcing roles on every protected API route was not identified. Therefore, the implementation can be described as providing authentication and separate role-based application views, but comprehensive server-side authorization for all API operations should not be claimed. A future improvement is to add token-verification and role-checking middleware to protected routes.

7. Features

7.1 List of Main Features

The Society Management Portal provides separate interfaces for residents and administrators. Its main features include:

1. **Resident registration and login** – Residents can register and sign in to access resident services.
2. **Administrator login** – Administrators sign in through a separate login flow to access management pages.
3. **Dashboard** – Residents and administrators have their own dashboard pages.
4. **Resident profile and management** – Residents can view their profile, while administrators have a page for managing resident information.
5. **Payment management** – Residents can view payment information and proceed to a payment checkout page. Administrators have a separate payments page.
6. **Visitor management** – Residents can use the visitor feature, and administrators can access visitor records.
7. **Complaint management** – Residents can submit complaints, and administrators can access the complaints page.
8. **Amenity management** – Residents can view amenity information, while administrators have an amenity management page.
9. **Notice management** – Residents can view notices, and administrators can access the notices page.
10. **Polls** – Residents can view and participate in polls, while administrators have a page for managing polls.
11. **Notifications** – Residents can view notifications, and administrators have a separate notifications page.

7.2 How Each Feature Works

1. Resident Registration and Login

User perspective: A resident registers or enters their login details to access the portal. After a successful login, they can open resident pages such as their dashboard, profile, payments, and society services.

Technical description: The React front end collects the required form data and sends it to the authentication API. The Node.js and Express back end processes the request. Passwords are checked using `bcrypt`; after successful login, the server creates a JSON Web Token (JWT) with a one-day expiry. Login-related information is stored in the browser's `localStorage`.

2. Administrator Login

User perspective: An administrator signs in through the administrator login page to access the management interface.

Technical description: The login details are sent to the authentication API. The back end checks the account and its administrator role, verifies the password using `bcrypt`, and returns a JWT after successful authentication. The front end then provides access to the administrator pages.

3. Dashboards

User perspective: The dashboard gives each type of user an entry point to the portal's features. Residents use the resident dashboard, while administrators use the administrator dashboard.

Technical description: React Router maps the dashboard URLs to their respective React pages. The pages render the relevant interface for the resident or administrator area.

4. Resident Profile and Management

User perspective: Residents can open their profile page to view their resident information. Administrators can use the resident management page to work with resident records.

Technical description: The front end provides separate resident profile and administrator resident pages. Data requests are handled by the resident API functions and corresponding back-end routes and controllers.

5. Payment Management

User perspective: Residents can view payment-related information and open the payment checkout page. Administrators can access the payments page to manage payment records.

Technical description: Payment-related pages use the payment API functions to communicate with the server. The front end displays the data returned by the back end. The project includes a checkout page; this description does not assume an external payment gateway is integrated.

6. Visitor Management

User perspective: Residents can use the visitor feature to provide visitor information. Administrators can open the visitor page to review and manage visitor records.

Technical description: Visitor information is handled through the visitor API, Express routes, and back-end controller. The front end sends and displays visitor data through these requests.

7. Complaint Management

User perspective: Residents can submit a complaint about a society-related issue. Administrators can access the complaints page to review and manage complaints.

Technical description: Complaint forms and records are connected to the complaint API. The Express back end routes requests to the complaint controller, which handles the required data operations.

8. Amenity Management

User perspective: Residents can view information about society amenities. Administrators can access the amenity page to manage amenity information.

Technical description: The amenity feature uses its own front-end API functions and corresponding back-end routes and controller to exchange amenity data.

9. Notice Management

User perspective: Residents can read notices published for the society. Administrators can use the notices page to manage notice information.

Technical description: Notice data is requested through the notice API and processed by the corresponding Express route and controller. The resident-facing page displays notices returned by the server.

10. Polls

User perspective: Residents can view available polls and participate. Administrators can access the polls page to manage poll information.

Technical description: Poll-related requests are handled by the poll API and its corresponding back-end route and controller. The front end displays poll data and submits user actions through the API.

11. Notifications

User perspective: Residents can open the notifications page to view messages or updates available in the portal. Administrators have a separate notifications page.

Technical description: The notification API communicates with the Express back end. The corresponding route and controller handle notification requests, and the React pages display the returned information.

Feature Access Note

The application provides separate resident and administrator login flows and interfaces. In the supplied project code, a shared back-end middleware for verifying JWTs and enforcing roles on every protected API route was not identified. Therefore, the report should describe the separate role-based views without claiming that every API operation has server-side role enforcement.

8. Testing

Testing is used to check that the Society Management Portal's features work as intended and that the front end communicates correctly with the back end. The tests in this chapter cover the main resident and administrator workflows, form validation, and data exchange between the React application, Express API, and MySQL database.

8.1 Functional Testing of the Front End and Back End

Functional testing checks whether each feature produces the expected behavior when a user interacts with it. Front-end checks focus on pages, forms, navigation, and displayed feedback. Back-end checks focus on processing requests and returning appropriate responses.

The following areas should be checked:

- Resident registration and login, including incorrect credentials.
- Administrator login and access to administrator pages.
- Navigation between dashboards and feature pages.
- Resident profile and resident management pages.
- Payment information and the checkout page.
- Visitor, complaint, amenity, notice, poll, and notification features.
- Form submission, required fields, and handling of unsuccessful requests.

8.2 Integration Testing

Integration testing checks that connected parts of the application work together. In this portal, this includes checking that a React page sends the correct request to its API function, that the Express route and controller process it, and that the response is displayed correctly by the page.

Examples include submitting a resident registration form and checking that the result is handled by the application, or opening a notices page and checking that the page displays the response received from the server. Similar checks apply to payments, visitors, complaints, amenities, polls, and notifications.

8.3 Testing Approach and Environment

The project's front end and back end can be tested together in the local development environment. The React application sends requests to the Express API configured at `http://localhost:5000/api`, and the server uses MySQL for persistent data.

Testing should be performed through the application's resident and administrator interfaces. For each workflow, enter valid and invalid data where applicable, submit the form or open the relevant page, and check the resulting screen and feedback. Postman was not used for this project and is therefore not included as a testing tool.

8.4 Test Cases and Results

The table below provides test cases for the main workflows. Record the actual result after running each test.

Test ID	Feature	Test action	Expected result	Execution result
TC-01	Resident login	Enter valid resident credentials and sign in	Login succeeds and the resident area opens	Record after testing
TC-02	Resident login	Enter incorrect credentials	Login is rejected and feedback is shown	Record after testing
TC-03	Administrator login	Enter valid administrator credentials and sign in	Login succeeds and the administrator area opens	Record after testing
TC-04	Navigation	Open a resident or administrator feature page	The selected page opens in the correct interface	Record after testing
TC-05	Resident profile	Open the resident profile page	The page displays the available resident information	Record after testing
TC-06	Payments	Open the payments page and proceed to checkout	Payment information or checkout page is displayed	Record after testing
TC-07	Visitors	Submit visitor information	The application handles the submission and displays an appropriate response	Record after testing
TC-08	Complaints	Submit a complaint	The application handles the complaint and displays an appropriate response	Record after testing
TC-09	Amenities	Open the amenities page	Available amenity information is displayed	Record after testing
TC-10	Notices	Open the notices page	Available notice information is displayed	Record after testing
TC-11	Polls	Open the polls page and submit a response, if available	The application handles the poll action and displays an appropriate response	Record after testing

TC-12	Notifications	Open the notifications page	Available notification information is displayed	Record after testing
TC-13	API or server error	Open a feature when its request cannot be completed	The page handles the unsuccessful request without silently displaying incorrect data	Record after testing

After executing the cases, replace “Record after testing” with the observed outcome, such as **Pass** or **Fail**, and add a brief note for any failure. This keeps the report’s results aligned with the tests actually performed.

9. Deployment

Deployment makes the Society Management Portal available in its intended environment. The application consists of a React front end, a Node.js and Express back end, and a MySQL database. The following sections describe the steps and configuration needed to run the application locally and the hosting arrangement suitable for production.

9.1 Steps to Deploy

The portal can be set up in the following order:

1. **Prepare the environment:** Install Node.js and MySQL on the development or deployment machine.
2. **Set up the database:** Create the MySQL database and apply the project's database schema and required initial data.
3. **Configure the back end:** Set the database connection details and other required settings in the back-end configuration. Install the back-end dependencies and start the Express server.
4. **Configure the front end:** Set the API address to point to the running back-end server. Install the front-end dependencies and start the React application or create its production build.
5. **Verify the application:** Open the front end and check that login, navigation, and the main resident and administrator features can communicate with the API and database.

During local development, the front end is configured to communicate with the API at `http://localhost:5000/api`. When deploying to another environment, this address must be changed to the address of the deployed back end.

9.2 Environment Configuration

The application requires configuration for the front end, back end, and database. The back end must be able to connect to MySQL using the correct host, database name, username, and password. These values should match the database environment where the application is running.

The front end must use the correct API base address for its environment. The local development address points to the machine running the Express server. In a hosted environment, it should point to the deployed API.

Development and production settings should be kept separate so local addresses and development credentials are not used in a public deployment. Database credentials and token-signing secrets should be stored securely and excluded from publicly accessible source files.

9.3 Hosting of Front End and Back End

For local development, the React front end and the Express back end run as separate services, while MySQL stores application data. The front end sends API requests to the back end, which processes them and accesses the database.

For production hosting, the React production build can be served from a static web host. The Express application requires a host that supports Node.js, and the MySQL database must be available to that server over a permitted connection. The deployed back end should allow requests from the front-end domain and use secure HTTPS connections.

No cloud hosting or production deployment configuration was identified in the supplied project materials. Therefore, this chapter describes the deployment arrangement and steps; it should not state that the portal has already been published online unless that deployment has been completed.

10. Challenges and Limitations

10.1 Issues Faced During Development

One challenge was connecting the React front end to the Node.js and Express back end. Each feature needed to send the appropriate request and display the returned data or feedback. Coordinating this flow across residents, payments, visitors, complaints, amenities, notices, polls, and notifications required organizing the application feature by feature.

Another challenge was supporting two types of users. Residents and administrators need different pages and workflows, so the application needed separate login flows and interfaces. Handling form data and page state consistently was also important, especially when a request was loading or returned an error.

The application also depends on a MySQL database to store its information. The back end must use the database records associated with each feature and return data in a form the front end can display.

10.2 Solutions Applied

The application's code is organized into feature-specific front-end API functions and back-end routes and controllers. This structure keeps requests for residents, payments, visitors, complaints, amenities, notices, polls, and notifications grouped by function.

Separate resident and administrator login flows and pages provide different interfaces for the two user types. React Router maps application paths to the relevant pages, while React state and effects support form handling and data display.

For login, the back end checks passwords using `bcrypt` and creates a JSON Web Token after successful authentication. The front end stores login-related information in browser storage for use by the application interface.

10.3 Current Limitations or Known Bugs

The supplied project code does not show shared back-end middleware that verifies JWTs and enforces user roles across every protected API route. The separate login flows and interfaces provide user separation in the application, but should not be described as complete server-side authorization.

The project is configured for local development, with the front end communicating with an API at `http://localhost:5000/api`. No production hosting configuration was identified in the supplied materials.

The payment feature includes payment-related pages and a checkout page. An external payment gateway integration was not identified in the supplied project materials. Automated test results were also not provided, so this report does not claim that the listed workflows have passed formal testing.

11. Future Enhancements

11.1 Planned Features

The portal could be improved by adding stronger access control for resident and administrator operations. This would include server-side JWT verification and role checks on protected API routes.

Other planned improvements could include:

- Adding an external payment gateway for online transactions.
- Providing residents with a way to track complaint status and responses.
- Adding search, filtering, and sorting to administrator pages.
- Improving notification delivery for important society updates.
- Adding reports for payments, complaints, visitors, and amenity usage.
- Improving mobile layouts for residents who access the portal on phones.

11.2 Possible Integrations or Optimizations

The application could be deployed to production by hosting the React build, Node.js and Express API, and MySQL database in suitable production environments. The front end would need to use the deployed API address, and production traffic should use HTTPS.

Further improvements could include input validation on both the front end and back end, clearer error handling, database indexing for frequently accessed records, and automated tests for the main user workflows. These changes would help improve reliability, security, and ease of maintenance as the portal grows.

12. Conclusion

12.1 Summary of the Project

The Society Management Portal is a web application designed to support common residential society activities. It provides separate resident and administrator interfaces and includes features for resident information, payments, visitors, complaints, amenities, notices, polls, and notifications.

The front end is built with React, the back end uses Node.js and Express, and MySQL is used for data storage. The front end communicates with the back end through feature-specific APIs.

12.2 What Was Achieved

The project brings several society-management functions together in one portal. Residents can access relevant services through the resident interface, while administrators have pages for managing society information and activities. The application also demonstrates how a front end, API layer, server-side logic, and relational database can work together.

12.3 Skills Learned During Development

Developing the portal provided experience with React components, routing, forms, and state management. It also involved creating API requests with `fetch`, organizing routes and controllers in Express, and connecting a Node.js back end to MySQL.

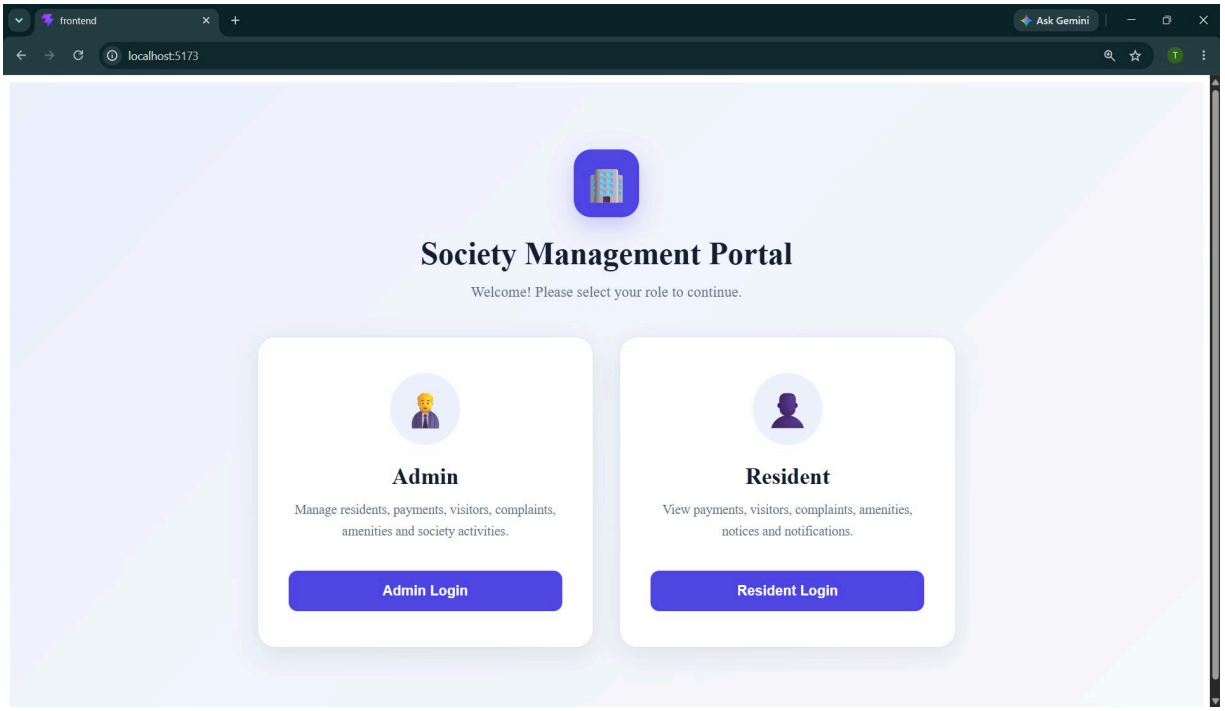
The project provided practical experience with login flows, password hashing using `bcrypt`, and JWT creation. It also highlighted the importance of testing feature workflows, validating user input, protecting API routes, and configuring an application appropriately before production deployment.

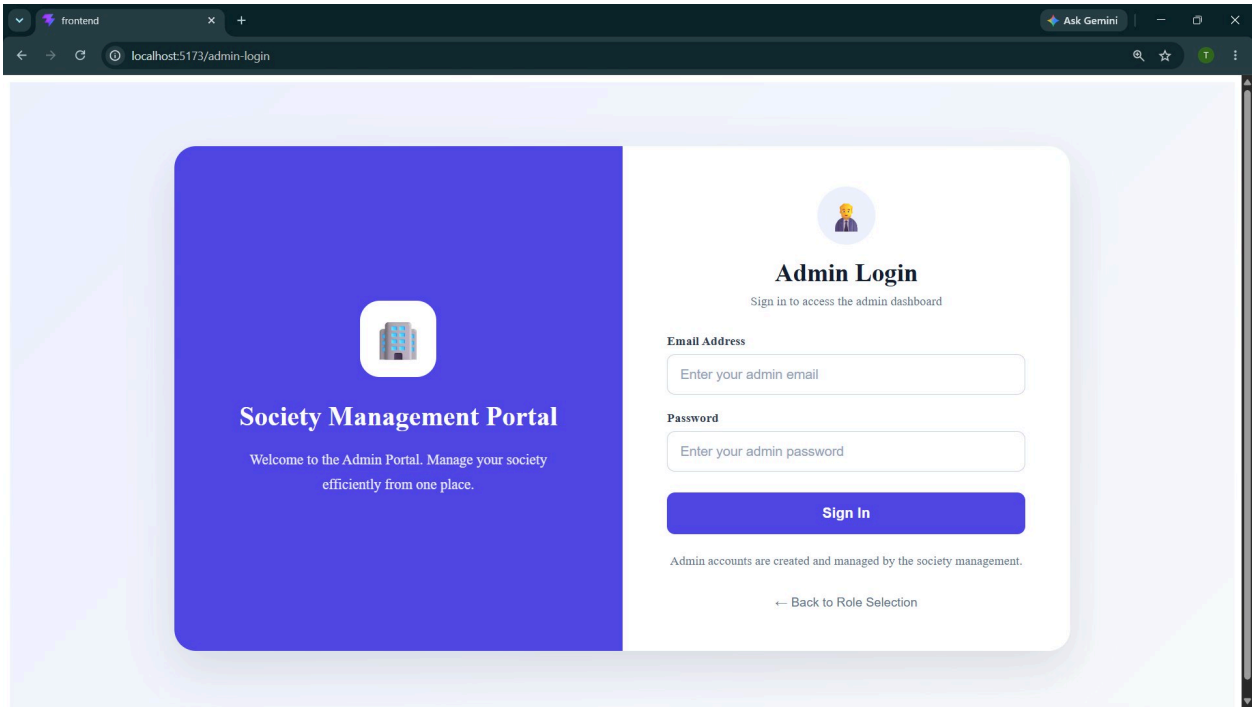
13. References

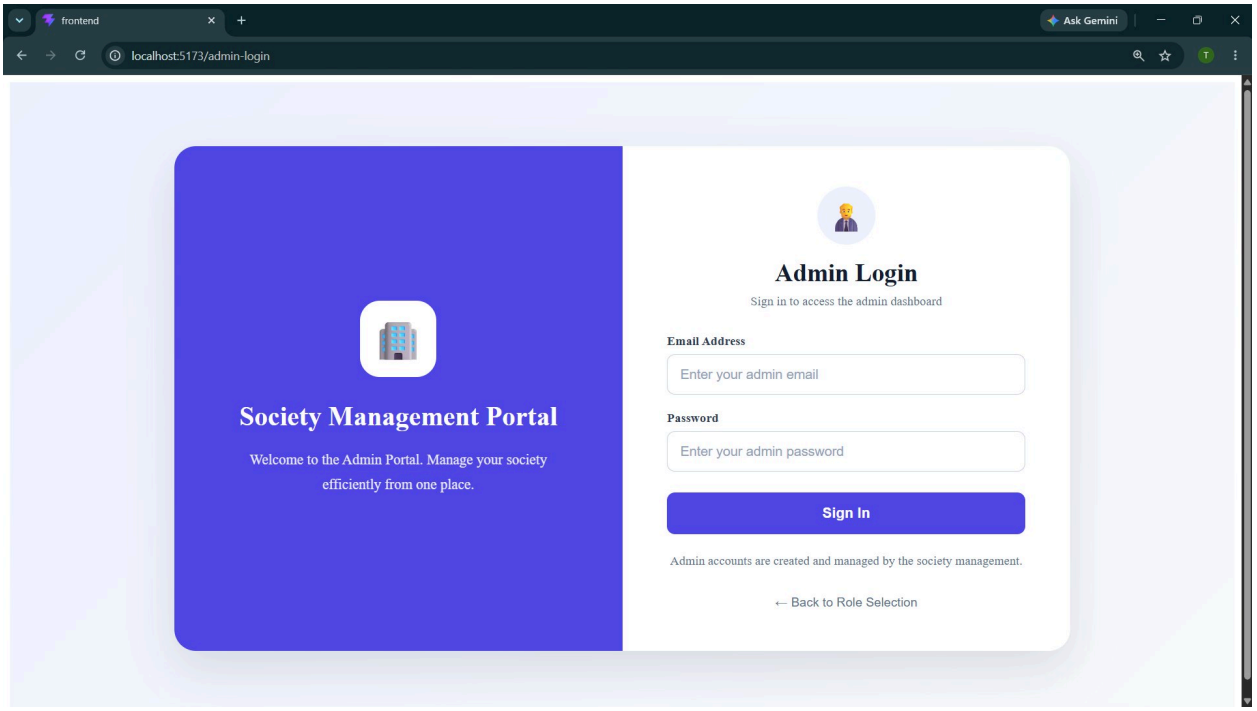
1. React Team, “React Documentation.” [Online]. Available: https://react.dev/learn. [Accessed: 2-Oct-2026].
2. OpenJS Foundation, “Node.js Documentation.” [Online]. Available: https://nodejs.org/docs/latest/api/. [Accessed: 2-Oct-2026].
3. OpenJS Foundation, “Express.js: Routing.” [Online]. Available: https://expressjs.com/en/guide/routing/. [Accessed: 2-Oct-2026].
4. Oracle, “MySQL 8.4 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/. [Accessed: 2-Oct-2026].
5. React Router, “React Router Documentation: Declarative Routing.” [Online]. Available: https://reactrouter.com/start/declarative/routing. [Accessed: 2-Oct-2026].
6. MDN Web Docs, “Fetch API.” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API. [Accessed: 2-Oct-2026].
7. IETF, “RFC 7519: JSON Web Token (JWT).” [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519. [Accessed: 2-Oct-2026].
8. kelektiv, “node.bcrypt.js: bcrypt for Node.js.” [Online]. Available: https://github.com/kelektiv/node.bcrypt.js/. [Accessed: 2-Oct-2026].

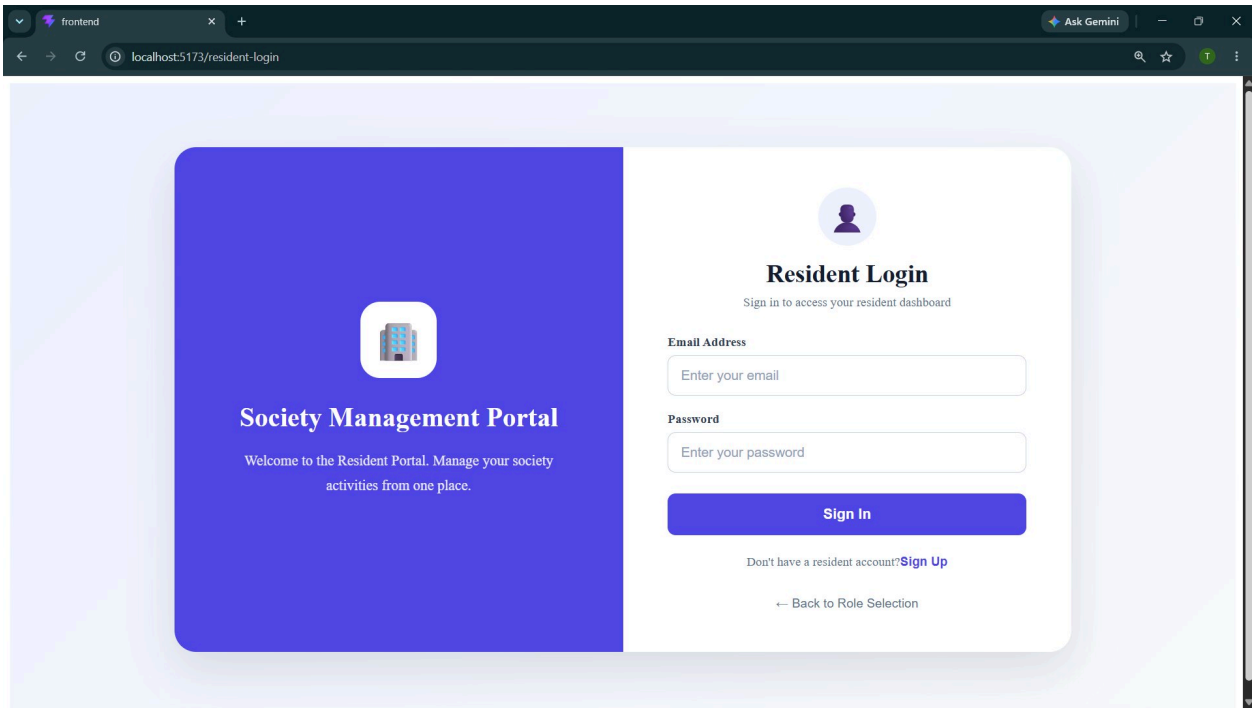
14. Appendices

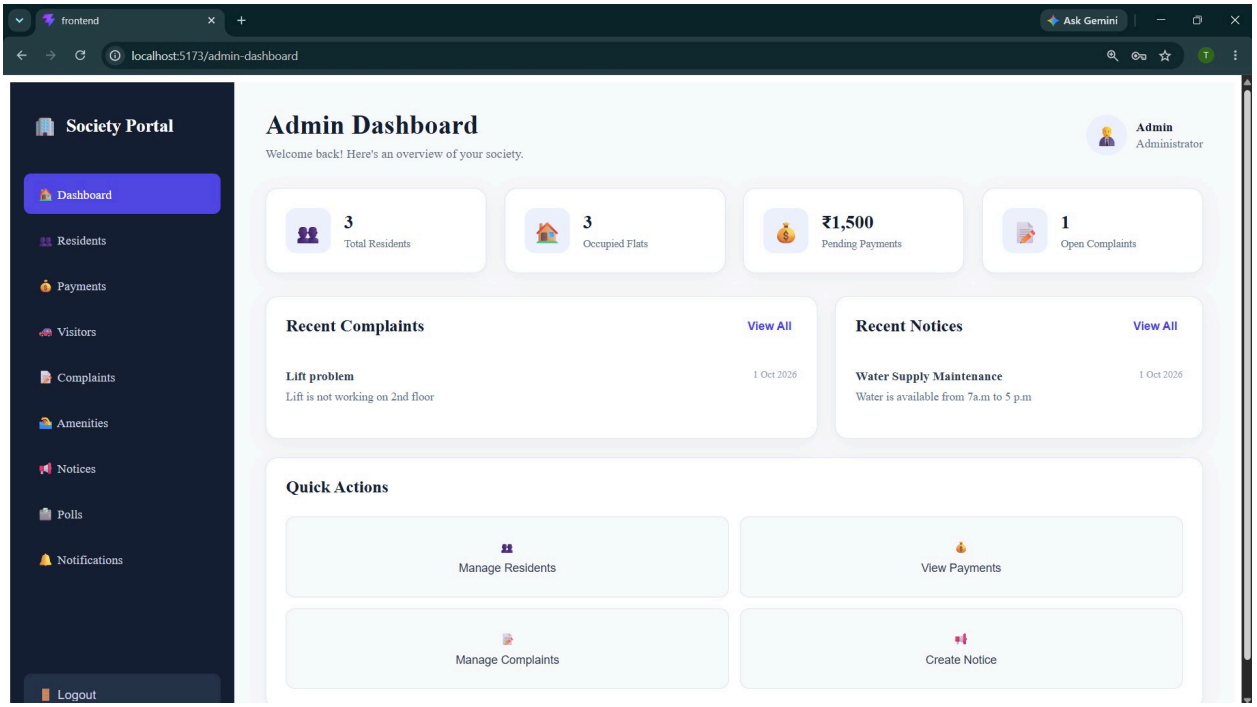
Screenshots of the Website:

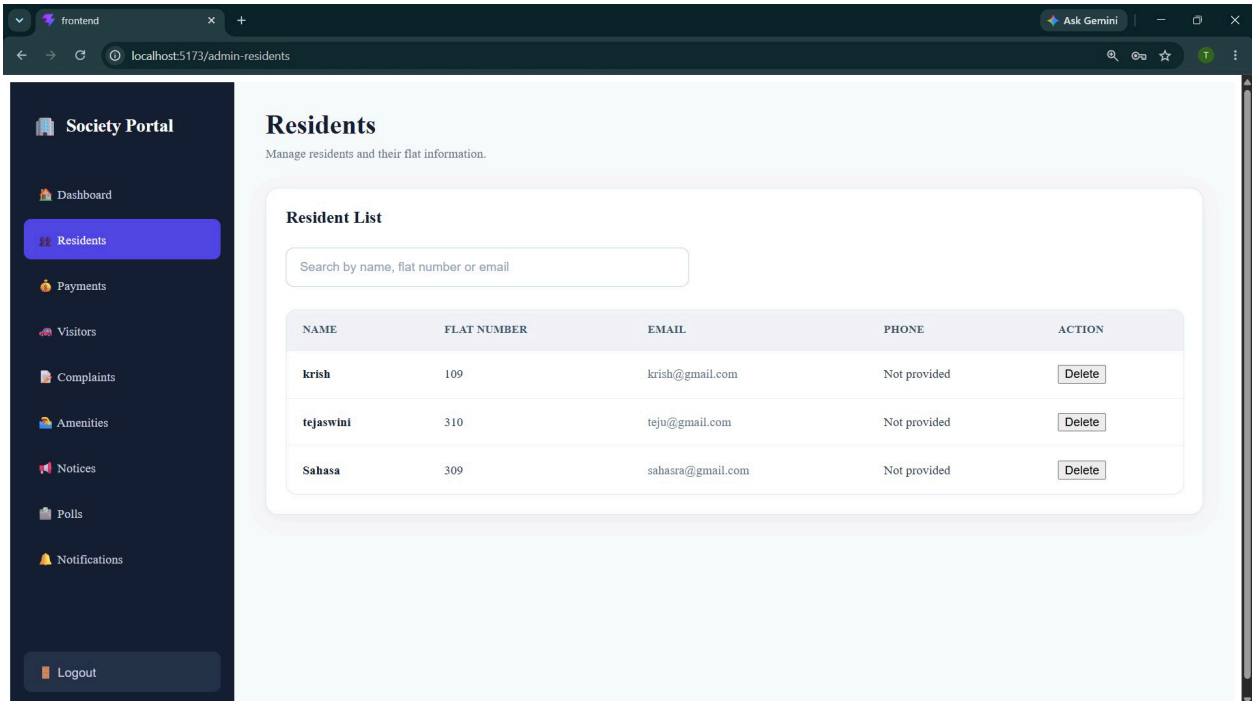


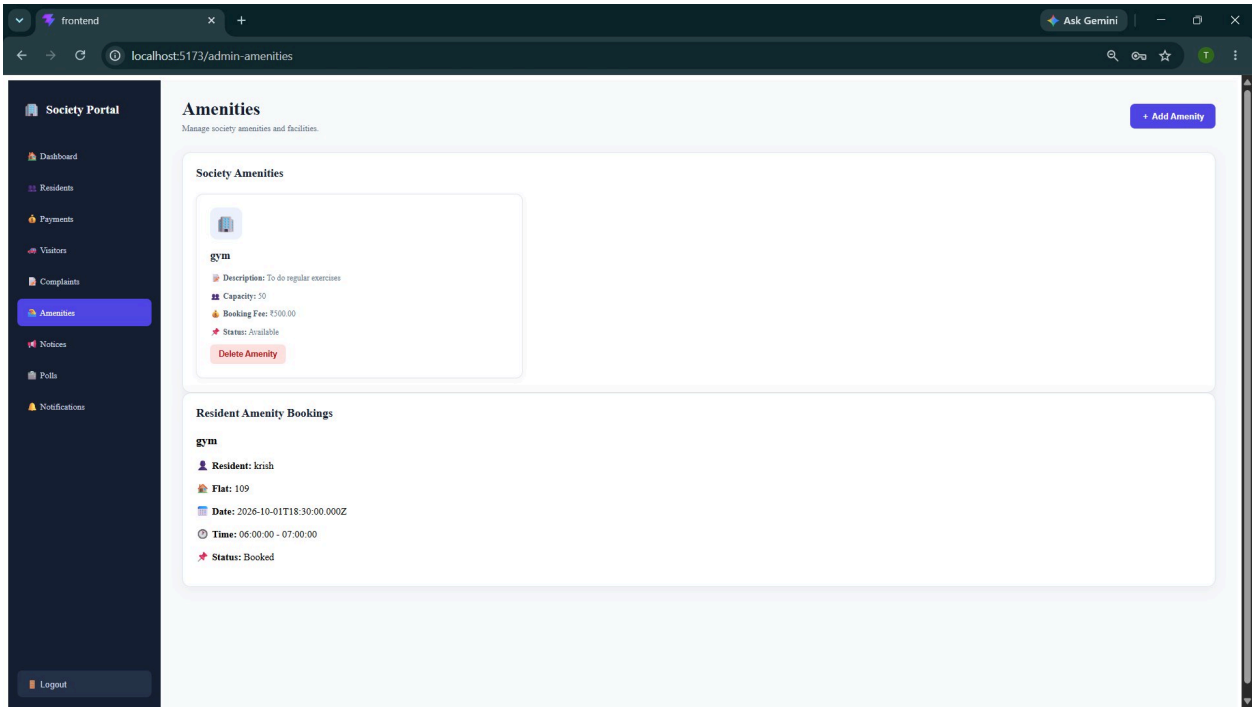



frontend
Ask Gemini
localhost:5173/admin-login
Society Management Portal
Welcome to the Admin Portal. Manage your society efficiently from one place.
Admin Login
Sign in to access the admin dashboard
Email Address
Enter your admin email
Password
Enter your admin password
Sign In
Admin accounts are created and managed by the society management.
← Back to Role Selection


frontend
Ask Gemini
localhost:5173/resident-login
Society Management Portal
Welcome to the Resident Portal. Manage your society activities from one place.
Resident Login
Sign in to access your resident dashboard
Email Address
Enter your email
Password
Enter your password
Sign In
Don't have a resident account?Sign Up
← Back to Role Selection


frontend
Ask Gemini
localhost:5173/admin-dashboard
Society Portal
Dashboard
Residents
Payments
Visitors
Complaints
Amenities
Notices
Polls
Notifications
Logout
Admin Dashboard
Welcome back! Here's an overview of your society.
Admin
Administrator
3
Total Residents
3
Occupied Flats
₹1,500
Pending Payments
1
Open Complaints
Recent Complaints
View All
Lift problem
1 Oct 2026
Lift is not working on 2nd floor
Recent Notices
View All
Water Supply Maintenance
1 Oct 2026
Water is available from 7a.m to 5 p.m
Quick Actions
Manage Residents
View Payments
Manage Complaints
Create Notice


frontend
Ask Gemini
localhost:5173/admin-residents
Society Portal
Dashboard
Residents
Payments
Visitors
Complaints
Amenities
Notices
Polls
Notifications
Logout
Residents
Manage residents and their flat information.
Resident List
Search by name, flat number or email
NAME
FLAT NUMBER
EMAIL
PHONE
ACTION
krish
109
krish@gmail.com
Not provided
Delete
tejaswini
310
teju@gmail.com
Not provided
Delete
Sahasa
309
sahasra@gmail.com
Not provided
Delete


frontend
Ask Gemini
localhost:5173/admin-amenities
Society Portal
Dashboard
Residents
Payments
Visitors
Complaints
Amenities
Notices
Polls
Notifications
Logout
Amenities
Manage society amenities and facilities.
+ Add Amenity
Society Amenities
gym
Description: To do regular exercises
Capacity: 50
Booking Fee: ₹500.00
Status: Available
Delete Amenity
Resident Amenity Bookings
gym
Resident: krish
Flat: 109
Date: 2026-10-01T18:30:00.000Z
Time: 06:00:00 - 07:00:00
Status: Booked

Sample Code Snippets:

backend > config > JS db.js > ...

```
const mysql = require("mysql2");

require("dotenv").config();

const db = mysql.createConnection({
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
    port: process.env.DB_PORT
});

db.connect((err) => {
    if (err) {
        console.error("MySQL connection failed:", err.message);
        return;
    }

    console.log("MySQL database connected successfully!");
});

module.exports = db;
```



```
                        "Invalid email or password"
                });
            }

            // Create JWT
            const token = jwt.sign(
                {
                    userId: user.user_id,
                    residentId: user.resident_id,
                    role: user.role_name
                },
                process.env.JWT_SECRET,
                {
                    expiresIn: "1d"
                }
            );

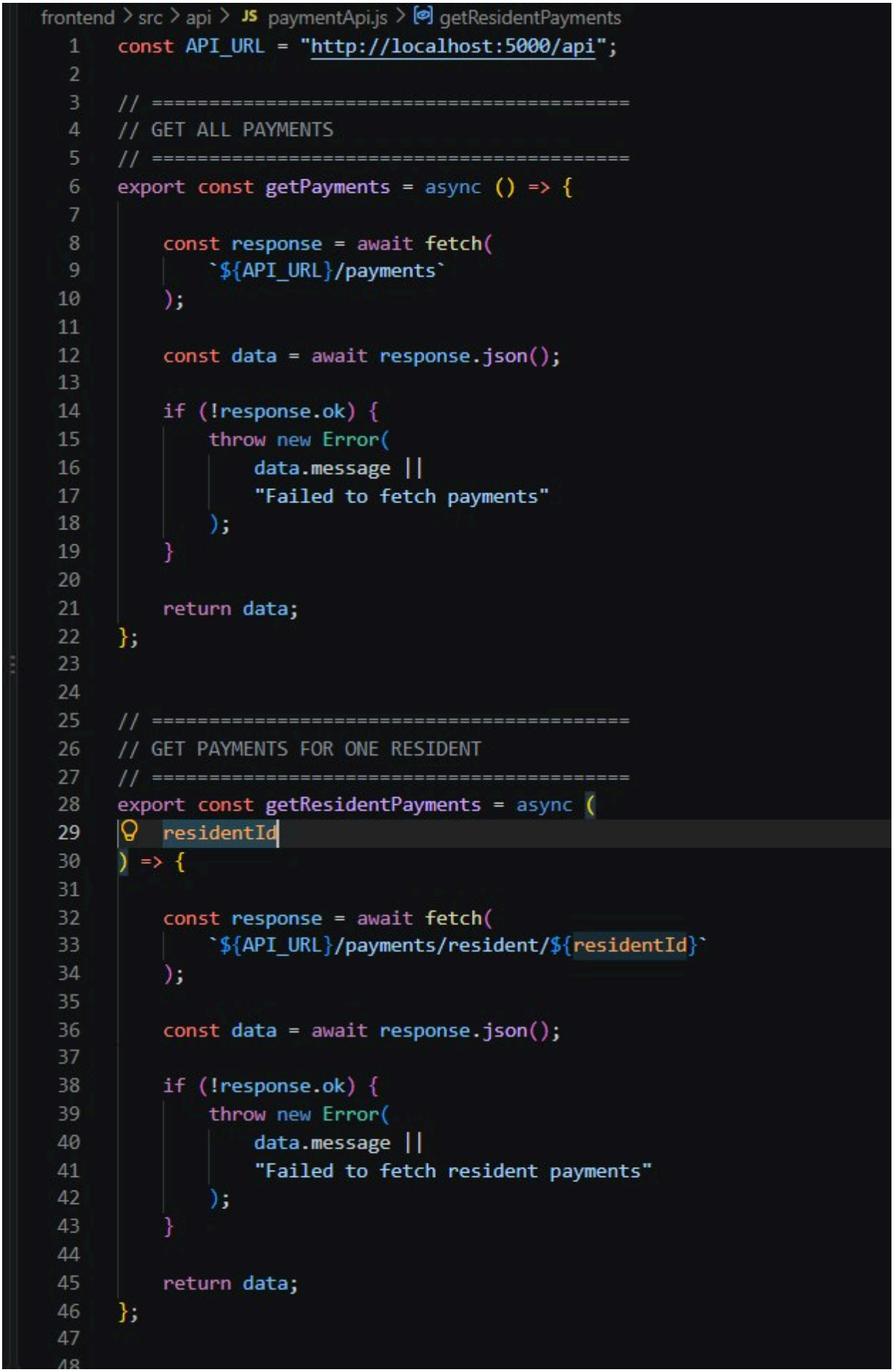
            // Send response
            return res.status(200).json({
                message: "Resident login successful",

                token,

                resident: {
                    userId: user.user_id,
                    residentId: user.resident_id,
                    name: user.name,
                    email: user.email,
                    flatNumber: user.flat_number,
                    phone: user.phone,
                    role: user.role_name
                }
            });
        }
    );
} catch (error) {
    console.error(
        "Login error:",
        error
    );

    return res.status(500).json({
        message: "Server error"
    });
}
};
```


residentApi.js
residentRoutes.js
residentController.js
db.js
frontend > src > api > residentApi.js > ...
const API_URL = "http://localhost:5000/api";
export const getResidents = async () => {
const response = await fetch(`${API_URL}/residents`);
if (!response.ok) {
throw new Error("Failed to fetch residents");
}
return response.json();
};
// DELETE RESIDENT
export const deleteResident = async (residentId) => {
const response = await fetch(
`${API_URL}/residents/${residentId}`,
{
method: "DELETE"
}
);
if (!response.ok) {
throw new Error("Failed to delete resident");
}
return response.json();
};


frontend > src > api > JS paymentApi.js > getResidentPayments
const API_URL = "http://localhost:5000/api";
// ==
// GET ALL PAYMENTS
// ==
export const getPayments = async () => {
const response = await fetch(
`${API_URL}/payments`
);
const data = await response.json();
if (!response.ok) {
throw new Error(
data.message ||
"Failed to fetch payments"
);
}
return data;
};
// ==
// GET PAYMENTS FOR ONE RESIDENT
// ==
export const getResidentPayments = async (
residentId
) => {
const response = await fetch(
`${API_URL}/payments/resident/${residentId}`
);
const data = await response.json();
if (!response.ok) {
throw new Error(
data.message ||
"Failed to fetch resident payments"
);
}
return data;
};

Installation / Setup Instructions:

1. Prerequisites

Install the following on the computer:

- Node.js and npm
- MySQL Server
- MySQL Workbench
- Visual Studio Code
- Google Chrome or another modern web browser
- Git (if the project is obtained from GitHub)

2. Obtain the Project

Download or clone the **Society Management Portal** project from the project repository.

If it is provided as a ZIP file:

- Extract the ZIP file.
- Open the extracted project folder in Visual Studio Code.

3. Set Up the Database

1. Open **MySQL Workbench**.
2. Create the database:

```
CREATE DATABASE society_management;
```

3. Select the `society_management` database.
4. Execute the project's SQL/table creation scripts.
5. Ensure the required tables and relationships are created.

4. Configure the Backend

1. Open the backend folder in VS Code.
2. Open the terminal.
3. Install the backend dependencies:

```
npm install
```

4. Configure the `.env` file with the MySQL database details, such as:

```
DB_HOST=localhost
DB_USER=your_mysql_username
DB_PASSWORD=your_mysql_password
DB_NAME=society_management
DB_PORT=3306
```

5. Start the backend server using the project's configured start command, for example:

```
node server.js
```

The backend runs on:

http://localhost:5000

and the API base URL is:

http://localhost:5000/api

5. Set Up the Frontend

1. Open the frontend folder in VS Code.
2. Open a terminal in the frontend directory.
3. Install dependencies:

npm install

4. Start the React development server:

npm run dev

5. Open the URL displayed by Vite, normally:

http://localhost:5173

6. Run the Application

Make sure **both servers are running**:

- Frontend → `localhost:5173`
- Backend → `localhost:5000`

Then open the frontend URL in Chrome.

The user can select either **Admin Login** or **Resident Login** from the Role Selection page.

7. Important Configuration Note

The database credentials and other sensitive configuration values should be kept in the `.env` file and should **not be uploaded to GitHub**.

User Guide / User Manual:

The Society Management Portal provides separate access for **Administrators** and **Residents**. The following steps describe the basic procedure for using the system.

1. Accessing the Application

1. Start the backend server and frontend application.
2. Open the application in a web browser.
3. The **Role Selection** page is displayed.
4. Select either **Admin** or **Resident** according to the user's role.

2. Admin Login

1. Select **Admin Login** from the Role Selection page.
2. Enter the registered admin email address.
3. Enter the password.
4. Click **Sign In**.
5. After successful authentication, the Admin Dashboard is displayed.

3. Admin Dashboard

The Admin Dashboard provides access to the major society-management operations. The administrator can manage and monitor information related to:

- Residents
- Payments
- Visitors
- Complaints
- Amenities
- Notices
- Polls
- Notifications

4. Resident Login / Registration

1. Select **Resident Login** from the Role Selection page.
2. Existing residents can enter their registered email and password and click **Sign In**.
3. New residents can select **Sign Up** and provide the required registration details.
4. After successful authentication, the Resident Dashboard is displayed.

5. Resident Dashboard

The Resident Dashboard allows residents to access the services provided by the portal. Depending on the available features, residents can:

- View and manage their profile information.
- View payment-related information.
- Manage or view visitor information.
- Submit and track complaints.
- View available amenities.
- View society notices.

- Participate in polls.
- View notifications.

6. Managing Complaints

1. Open the **Complaints** section.
2. Enter the required complaint details.
3. Submit the complaint.
4. The complaint can be viewed and tracked through the portal.
5. Administrators can review and manage submitted complaints.

7. Managing Visitors

1. Open the **Visitors** section.
2. Enter the required visitor information.
3. Submit/save the visitor details.
4. The visitor information can be viewed through the portal.

8. Payments

1. Open the **Payments** section.
2. View the available payment-related information.
3. Residents can access their payment records.
4. Administrators can manage or monitor payment information.

9. Notices and Notifications

Residents can open the **Notices** and **Notifications** sections to view information published by the society administration.

Administrators can create and manage notices and notifications so that important information can be communicated to residents.

10. Polls

1. Open the **Polls** section.
2. View the available polls.
3. Select the required option.
4. Submit the response.
5. The administrator can manage the polls through the Admin Portal.

11. Logout

After completing the required operations:

1. Click the **Logout** option.
2. The current session is terminated.
3. The user is redirected to the appropriate login/role-selection page.

12. Basic User Guidelines

- Users should use their own valid login credentials.

- Passwords should not be shared with other users.
- Required information should be entered accurately.
- Users should log out after completing their work.
- Administrators should verify information before making changes to society records

Geotag photos with guide:





Review forms with guide signatures: